team-summary-report

Team Summary Report — BlackCS

Project: ProPath — Career preparation & professional development **Team:** BlackCS **Members:** Lovinson Dieujuste, Malcolm Richards **Course milestone:** Final report · **Date:** April 30, 2026 **Repository:** https://github.com/Wisesofthemall/FocusFlow **Demo video (Milestone 3):** https://youtu.be/8M7t6lzVS9s

---

1. Executive summary

ProPath is a single-page web application that helps students and early-career professionals organize their job search. Users sign in, add and edit job applications with a status and a next-action date, see what's due this week on a unified dashboard, and browse trending remote jobs from a public external API — all backed by a stateless JWT-authenticated Spring Boot REST API and a persistent file-based H2 database. The MVP is shipped, every milestone deliverable is in the repository, and the test suite is green.

---

2. Architecture overview

2.1 Stack

Layer	Technology
Frontend	React 19, Vite 8, TypeScript, Axios
Backend	Spring Boot 3.3.5, Java 17, Spring Security, Spring Data JPA
Database	File-based H2 (`backend/data/propath.mv.db`)
Authentication	JWT (Bearer token, stateless), bcrypt password hashing
External API	RemoteOK jobs feed (server-side proxy via `WebClient`)
Build tooling	Maven (backend), Vite (frontend)
Test frameworks	JUnit 5 + MockMvc (backend), Vitest + RTL (frontend)

2.2 System architecture

```
┌──────────────────┐   HTTPS/CORS   ┌──────────────────────────┐   Bearer JWT   ┌──
│ React (Vite)     │ ──────────────▶ │ Spring Boot REST API     │ ──────────────▶ │ Se
│  :5173           │                 │  :8080                   │                 │ (U
│  - AuthContext   │                 │  - SecurityFilterChain   │                 │  C
│  - Axios w/      │                 │  - JwtAuthFilter         │                 └──
│    interceptor   │                 │  - Controllers (/api/)   │
└──────────────────┘                 └──────────────────────────┘
                                                  │
                                                  │  WebClient (reactive)
                                                  ▼
                                                                                   ┌
                                                                                   │
                                                                                   └
```

```
│ RemoteOK API     │
│ remoteok.com     │
└──────────────────┘
```

2.3 Key design decisions

- **Stateless JWT auth, not server sessions.** Tokens are issued on register/login, carried as `Authorization: Bearer <token>`, and verified by `JwtAuthenticationFilter` on every request. This keeps the backend horizontally scalable and removes the need for a session store.
- **`userId` derived from the principal, not the URL.** A mid-project refactor closed two ownership leaks where one user could read another user's applications by changing a path variable. All per-user endpoints now read `userId` from the authenticated `SecurityContext` and reject anything that doesn't match the resource owner.
- **External API is proxied, not exposed.** The browser never talks to RemoteOK directly. The Spring backend fetches the feed via `WebClient`, picks specific fields, and returns a small JSON payload. This avoids CORS gymnastics and keeps the upstream trust boundary inside the API tier.
- **File-based H2, not in-memory.** Data persists across server restarts, which is what makes the demo's "restart the backend, refresh the page, your application is still there" step possible. Tests run against a separate in-memory H2 instance via `src/test/resources/application.properties`, so dev data is never touched.
- **First-boot demo seeding.** A `CommandLineRunner` seeds `demo@propath.local` with three sample applications at varying urgencies the first time the server starts. A grader can sign in immediately and see the urgency sort doing real work.

---

3. Features delivered

3.1 MVP capabilities

Capability	Status	Notes
User registration & login	Shipped	Bcrypt-hashed passwords; JWT issued on success; demo account seeded on first boot.
Application CRUD	Shipped	`POST / GET / PUT / DELETE /api/applications`; ownership enforced from JWT principal.
Auto-sort by urgency	Shipped	Applications ordered by `nextActionDate`; the dashboard "due this week" card uses the same rule.
Dashboard	Shipped	Three cards: applications due this week, upcoming events, trending remote jobs.
External API integration	Shipped	RemoteOK jobs feed, fetched server-side and surfaced on the dashboard.
Persistence	Shipped	File-based H2 at `backend/data/propath.mv.db`; survives restarts.
Auth-guarded routes	Shipped	`ProtectedRoute` on the React side; Spring Security on the API side; both verified by tests.

3.2 Pages (React)

1. **Login** — email + password form, calls `POST /api/auth/login`, stores JWT, redirects to dashboard.
2. **Register** — same shape as Login, calls `POST /api/auth/register`.
3. **Dashboard** — three cards (urgency-sorted applications, upcoming events, RemoteOK jobs).
4. **Applications list** — sorted by `nextActionDate`; status badge; edit/delete actions.
5. **New Application** — form, validates required fields, redirects back to list on success.

3.3 Endpoints (Spring Boot)

Full table in `docs/endpoints.md`. Highlights:

Method	Path	Auth	Purpose
POST	`/api/auth/register`	No	Create account, return JWT.
POST	`/api/auth/login`	No	Authenticate, return JWT.
GET	`/api/users/me`	Yes	Return the authenticated user's profile.
GET	`/api/applications`	Yes	List the user's applications.
POST	`/api/applications`	Yes	Create an application.
GET	`/api/applications/{id}`	Yes	Read one (ownership-checked).
PUT	`/api/applications/{id}`	Yes	Update one (ownership-checked).
DELETE	`/api/applications/{id}`	Yes	Delete one (ownership-checked).
GET	`/api/jobs/trending`	Yes	Proxied RemoteOK feed.

---

4. Testing summary

4.1 Suite results

Suite	Framework	Tests	Result	Command
Backend (JUnit 5)	Spring Boot Test + MockMvc	5	5 passed, 0 failed	`cd backend && ./mvnw test`
Frontend (Vitest + RTL)	Vitest + React Testing Library	10	10 passed, 0 failed	`cd frontend && npm run test:run`

Total: 15 automated tests, all passing. Frontend coverage (V8): 62.92% statements / 46.34% branches across exercised files. Test-evidence screenshots are in `docs/test-evidence/screenshots/`.

4.2 What is covered

Backend (`backend/src/test/java/com/blackcs/propath/web/`):

- `ApplicationControllerTest` — full CRUD on `/api/applications`: create + read by id, update status + delete + verify gone (POST, GET, PUT, DELETE).
- `UserControllerTest` — `/api/users/me` returns 200 with a valid JWT and 401 without one. Proves `JwtAuthenticationFilter` is wired into the `SecurityFilterChain`.
- `ProPathApplicationTests` — Spring context smoke test.

Tests run against in-memory H2 via a separate test profile, so the file-based dev DB at `backend/data/propath.mv.db` is never touched.

Frontend (`frontend/src/`):

- `StatusBadge.test.tsx` — correct human label and status-specific class for each of the 5 `ApplicationStatus` values.
- `ProtectedRoute.test.tsx` — redirects to `/login` when no token; renders children when a token is present (exercises the `AuthContext` hydration race).
- `LoginPage.test.tsx` — form submit calls the auth API, stores JWT in `localStorage`, navigates to `/`; on API failure shows the error banner without storing a token.

4.3 Peer usability testing

Three peer testing sessions surfaced two issues that no automated test could have caught:

1. **The empty state on `/applications` doesn't tell brand-new users what to click first.** A user with zero applications sees a blank panel and has to guess that "+ New application" is the next step. Fix: a CTA card in the empty state.
2. **The delete button on the list page has no confirmation dialog.** One peer triggered an accidental delete during their session. For a tool that holds job-search data, this isn't a cosmetic UX issue — it's a real harm. Fix: a confirm modal before destructive actions.

Both items are on the punch list for the next iteration.

---

5. Future improvements

Highest-priority items

1. **Confirmation modal before delete** — accessibility / harm-reduction. Highest priority because the cost of an accidental delete is real and the fix is small.
2. **Empty-state CTA** — usability fix for new-user onboarding. Surface "+ New application" prominently when the list is empty.
3. **`JobApplicationResponse` DTO** — the API currently returns the JPA entity (with `@JsonIgnore` patching over the `user` field). Extract a response DTO so the entity stops leaking through the API. The need was made obvious by writing field-level test assertions.

Medium-priority items

4. **Migrate JWT from localStorage to `httpOnly` cookie** with CSRF protection. The README documents this as a known tradeoff; it's the right next step once the

milestone scope no longer constrains us.

5. **Tighten frontend coverage reporting.** Configure Vitest's `coverage.include` so coverage is reported across the whole `src/` tree, not just files imported by tests. As-is, `StatusBadge.tsx` doesn't even appear in the coverage table — the real number is lower than the headline 62.92% suggests. Honest tooling is a feature.
6. **Google Calendar OAuth integration.** Originally the planned external API; deferred to RemoteOK to fit Milestone 3 scope. Adding Calendar back in is the obvious feature direction for a v2.

Lower-priority / speculative

7. **Move from H2 to PostgreSQL** for any non-local deployment. Schema is JPA-portable; the change is mostly DDL and connection-string work.
8. **Disable the H2 console** in any non-dev profile.
9. **Move JWT secret to a managed secret store** rather than relying on the `JWT_SECRET` env var with a dev-only default.

---

6. Team roles and acknowledgments

6.1 Roles (per Milestone Document §7.1)

Member	Responsibility
Lovinson Dieujuste	Frontend (React), UI states, API client, demo polish; database schema and JPA entities; diagrams and wireframes; cross-stack implementation owner.
Malcolm Richards	Backend (Spring Boot), authentication, application REST APIs; security review.

6.2 Honest contribution note

The git history is authored entirely by Lovinson Dieujuste. That reflects the implementation reality: Lovinson owned cross-stack code. Malcolm contributed through design alignment and role-specific review (auth model, ERD, entity shape, security tradeoffs) and shared documentation work. The team has chosen to record this honestly here rather than to misrepresent the commit graph.

6.3 Acknowledgments

- **Course staff** — for the milestone cadence (Sprint 1 → Milestone 3 → Milestone 4) that forced us to ship the smallest demonstrable version of each capability rather than batch everything to the end.
- **Peer reviewers** — three peer-testing sessions surfaced two real harm-reduction issues (empty state, unconfirmed delete) that no automated test would have caught.
- **Open-source projects** — Spring Boot, React, Vite, TypeScript, Vitest, RTL, JUnit 5, H2, RemoteOK's public jobs feed.
- **AI tools (planning aid only).** ChatGPT and Claude (Opus 4.7) were used for ideation, architecture planning, and a best-practices audit, all logged with prompt summaries and influence notes in `docs/ProPath-Milestone-Document.md §5.2`. No

code in the repository was written by AI; every Java class, React component, config, test, and doc was authored, reviewed, and committed by BlackCS members.

---

7. Repository artifacts

Artifact	Path
Project README	README.md
Milestone document (proposal + design)	docs/ProPath-Milestone-Document.md
Endpoint table	docs/endpoints.md
ERD	docs/erd.md
API request/response examples	docs/api-examples.md
Wireframes (Figures 1–4)	docs/wireframes/
Architecture & DFD diagrams	docs/diagrams/
Test-evidence screenshots	docs/test-evidence/screenshots/
AI usage log	docs/ProPath-Milestone-Document.md §5.2
Demo video (Milestone 3)	https://youtu.be/8M7t6lzVS9s